Interrupt-Driven Client-Server System with GPIO Control

Project Purpose:

This project demonstrates an interrupt-driven system where external events trigger GPIO operations. A client-server architecture facilitates communication, with the client sending requests via a socket connection and the server processing them. The system ensures efficient and responsive hardware control through GPIO drivers.

Functionalities of the system:

- GPIO Interrupt Handling: The GPIO button (GPIO pin 535) triggers an interrupt when pressed. The interrupt handler sets a flag to indicate the button press.
- Device Operations: The device interacts with the GPIO button to manage interrupt events. It is opened and closed via file operations and reads the status of the interrupt flag.
- File Operations: The driver allows interaction with the GPIO button through basic file operations (open, close, read). A user can read the interrupt status from the driver.
- Server Communication: A server listens on a specified port, accepts connections, and can send commands to control an LED through a device file interface.
- Client Commands: The client sends commands like "on" or "off" to the server, which then writes the command to the device file to control the LED's state.

Operating Environment:

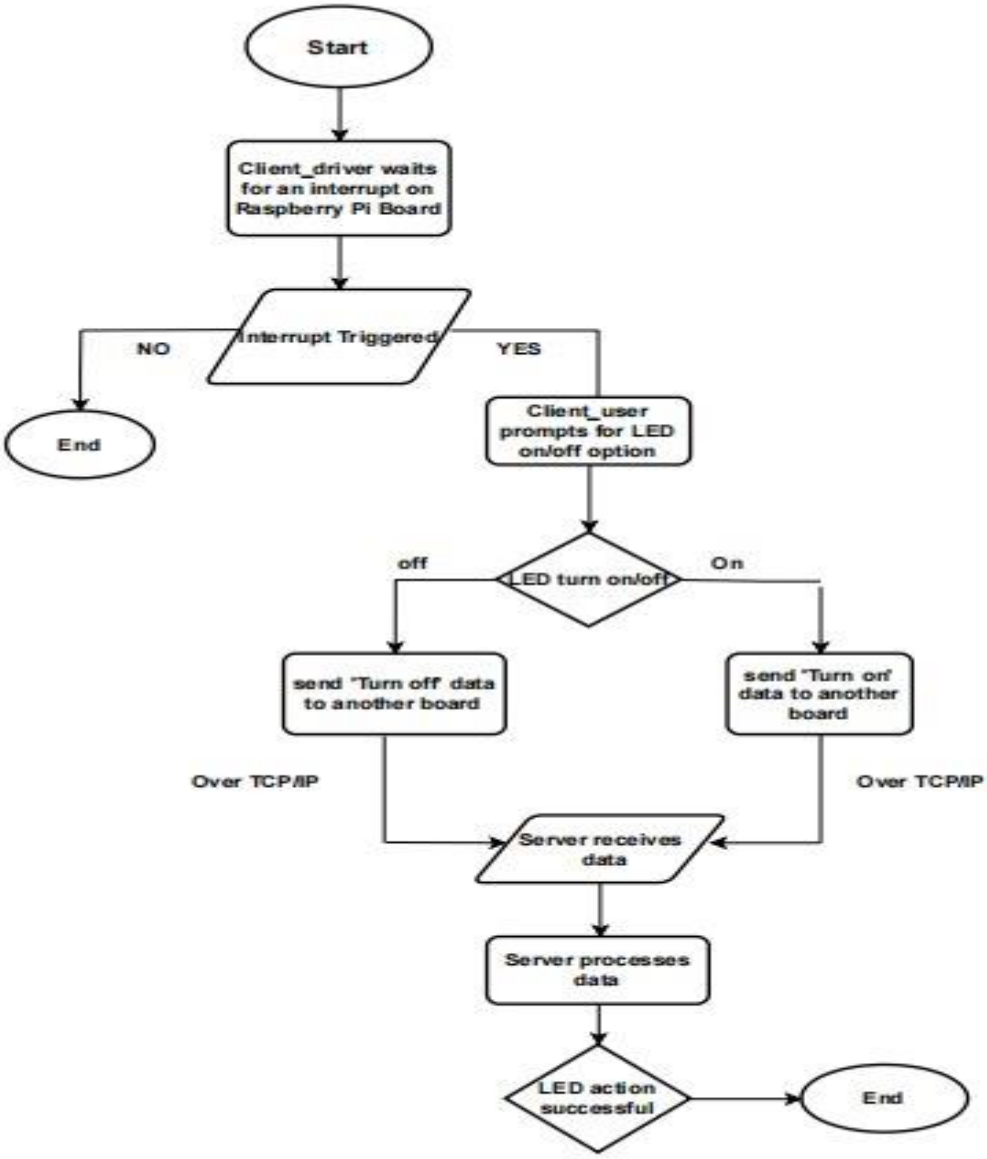
Client/Server System: Two Raspberry Pi boards, one as the client and the other as the server.

Operating System: Linux (Raspberry Pi OS).

Platform: Raspberry Pi (model 4 B).

Protocol: TCP/IP for file transfer**.**

Flow chart:



1. Start Block: The system begins its operation and waits for an interrupt to occur.

2. GPIO Interrupt Handler: When an interrupt occurs, it is handled by the GPIO interrupt handler. The interrupt may be triggered by an external event (e.g., button press or sensor input).

3. Setting Flag: After handling the interrupt, a flag is set to signal the client application that an event has occurred.

4. Client Application: The client application reads the flag and initiates communication with the server application. It establishes a TCP/IP connection to transfer data or commands.

5. Server Application: The server application receives data or commands from the client over the established TCP/IP connection. It processes the received information to determine the next action.

6. LED Control Signal: Based on the server's decision, it sends a control signal to the GPIO driver to turn the LED on or off.

7. GPIO Driver: The GPIO driver translates the control signal into an electrical signal that controls the LED's state.

8. LED On/Off: The LED's state is changed (turned on or off) based on the received control signal.

Conclusion:

The client establishes a TCP/IP connection with the server and sends a packet containing the command to turn the LED on or off. After sending the command, it may wait for a response from the server or terminate the connection based on the workflow.

The server listens for incoming client connections and processes requests as they arrive. Upon receiving a command, it interacts with the GPIO driver to perform the specified LED operation. The server ensures the action is completed and may send a response back to the client before waiting for the next request.